GROUP PROJECT TECHNICAL REPORT

AI Learning Platform with Intelligent Tutor & AI Command Terminal

Course: IT41073 - Service Oriented Computing
Project Title: IntelliLearn — Microservices LMS, RAG AI Tutor & AI Command Terminal
Author / Contributor: Chamod (30% Subsystem Lead & Microservices Architect)
Branch: `chamodDev`
Date: August 11, 2026

---

1. Executive Summary & Project Introduction

Modern online education requires scalable, resilient, and intelligent software platforms. The **IntelliLearn Platform** is a cloud-native Learning Management System (LMS) designed using a **Microservices Architecture** paired with an **AI-powered Intelligent Tutor** (RAG) and an **AI Command Terminal** (Phase 1 & Phase 2) for DevOps/Linux assistance.

Traditional monolithic LMS architectures suffer from single points of failure, scaling bottlenecks during exam peaks, and tightly coupled database schemas. In contrast, IntelliLearn decouples business logic into independent microservices (`user-service`, `course-service`, `quiz-service`, `progress-service`, `tutor-service`) fronted by a centralized **API Gateway** on port `8080`.

Students can enroll in courses, attempt practice assessments with automatic score feedback, track module progress percentages, query an AI Tutor grounded directly in uploaded course documents (`Java_OOP_Guide.pdf`, `Spring_Boot_Guide.pdf`), and utilize an **AI Command Terminal** to convert natural language into safe, explained CLI shell commands with automated error diagnosis, line-by-line flag deconstruction, and controlled process execution.

---

2. System Architecture & High-Level Design

2.1 Microservices Architecture Diagram

```
graph TD
    Client["React Frontend Client\n(Port 3000 / Nginx)\n• Dashboard & AI Command Terminal UI (Phase 2)"]

    subgraph Edge Layer
        Gateway["API Gateway\n(Port 8080)\n• Centralized Routing\n• Keycloak OAuth2 / JWT Validation\n• Redis IP Rate Limiting (/api/tutor/command/**)\n• Global CORS Policy"]
    end

    subgraph Microservices Layer
```

```
        UserService["User Management Service\n(Port 8081)\n• User Auth &
Profiles\n• MongoDB userdb"]
        CourseService["Course Management Service\n(Port 8082)\n• Course Syllabus &
Enrolment\n• MongoDB coursedb"]
        QuizService["Quiz & Assessment Service\n(Port 8083)\n• Quiz Player &
Evaluation\n• MongoDB quizdb (quiz_attempts)"]
        ProgressService["Learning Progress Service\n(Port 8084)\n• Progress
Analytics & Percentages\n• MongoDB progressdb"]
        TutorService["AI Tutor & AI Command Subsystem\n(Port 8085)\n• LLM Provider
Abstraction (Gemini/Mock)\n• Safety Engine (LOW/MED/HIGH/CRITICAL)\n• Controlled
Executor (5s Timeout Sandbox)\n• Flag Deconstruction Engine\n• Error Diagnoser &
RAG Engine\n• MongoDB tutordb (command_history)"]
    end

    subgraph Shared Infrastructure Layer
        MongoDB[(MongoDB Database 7.0\nPersistent Volume: mongodb-data)]
        Redis[(Redis Cache 7.0\nRate Limiting Key-Value)]
        Keycloak[(Keycloak IAM 25.0\nOAuth2 & JWT Issuer)]
    end

    Client -->|HTTPS / REST API| Gateway
    Gateway -->|Redis Rate Limiter| Redis
    Gateway -->|Validate JWT Token| Keycloak
    Gateway -->|Route /api/users, /api/auth| UserService
    Gateway -->|Route /api/courses| CourseService
    Gateway -->|Route /api/quizzes| QuizService
    Gateway -->|Route /api/progress| ProgressService
    Gateway -->|Route /api/tutor/**| TutorService

    UserService --> MongoDB
    CourseService --> MongoDB
    QuizService --> MongoDB
    ProgressService --> MongoDB
    TutorService --> MongoDB
```

---

3. Microservices Detailed Specifications

3.1 User Management Service (`user-service`)

- **Port**: `8081` | **Database**: MongoDB (`userdb`)
- **Endpoints**: `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/{id}`

3.2 Course Management Service (`course-service`)

- **Port**: `8082` | **Database**: MongoDB (`coursedb`)
- **Endpoints**: `GET /api/courses`, `GET /api/courses/{id}`, `POST /api/courses/{id}/enroll`, `POST /api/courses`

3.3 Quiz & Assessment Service (`quiz-service`)

- **Port**: `8083` | **Database**: MongoDB (`quizdb`, collection `quiz_attempts`)
- **Endpoints**: `GET /api/quizzes`, `POST /api/quizzes/{id}/submit`, `GET /api/quizzes/{quizId}/attempts/{userId}`, `POST /api/quizzes`

3.4 Learning Progress Service (`progress-service`)

- **Port**: `8084` | **Database**: MongoDB (`progressdb`)
- **Endpoints**: `GET /api/progress`, `GET /api/progress/{userId}`, `POST /api/progress`, `PUT /api/progress/{userId}/{courseId}`

3.5 AI Tutor & AI Command Terminal Subsystem (`tutor-service`)

- **Port**: `8085` | **Database**: MongoDB (`tutordb`, collections `tutor_interactions` & `command_history`)
- **Endpoints**:
  - `POST /api/tutor/ask`: RAG course document Q&A.
  - `POST /api/tutor/command/generate`: Translates natural language into CLI commands with risk classification.
  - `POST /api/tutor/command/explain`: Line-by-line tokenized CLI flag deconstruction.
  - `POST /api/tutor/command/execute`: Controlled safe process execution sandbox with 5-second timeout.
  - `POST /api/tutor/command/diagnose`: Diagnoses terminal error logs and suggests diagnostic commands.
  - `GET /api/tutor/command/history`: Fetches student command history from MongoDB (`command_history`).
  - `DELETE /api/tutor/command/history`: Clears all student command history.
  - `DELETE /api/tutor/command/history/{id}`: Deletes a single history record.

---

4. Empirical Test & Verification Matrix

Verification Test	Method / Endpoint	Expected Result	Status
Gateway Health	`GET /gateway/health`	`200 OK {"status":"Gatew`	**PASS**

Verification Test	Method / Endpoint	Expected Result	Status
		`ay is running"}`	
Quiz Persistence	`POST /api/quizzes/1/submit`	`200 OK` saved to MongoDB `quiz_attempts`	**PASS**
AI Command Generation	`POST /api/tutor/command/generate`	`200 OK "command":"find ~ -type f -size +100M"`	**PASS**
Command Flag Deconstruction	`POST /api/tutor/command/explain`	`200 OK` Tokenized breakdown of flags & arguments	**PASS**
Controlled Safe Execution	`POST /api/tutor/command/execute`	`200 OK status:"SUCCESS", exitCode:0, stdout`	**PASS**
Dangerous Command Safety Lock	`POST /api/tutor/command/execute` (`rm -rf`)	`200 OK status:"BLOCKED_HIGH_RISK"`	**PASS**
Error Diagnosis Engine	`POST /api/tutor/command/diagnose`	`200 OK` Diagnosis + suggested diagnostic commands	**PASS**
Single History Delete	`DELETE /api/tutor/command/history/{id}`	`204 No Content` Record deleted from MongoDB	**PASS**

---

5. Individual Contribution Matrix (Target: ~30%)

Subsystem / Feature	Contribution %	Author	Key Technical Work
AI Command Terminal Subsystem (Phase 1 & 2)	100%	Chamod	Designed CLI generation engine, flag deconstruction, DTOs, `CommandController`, and React terminal UI
Controlled Safe Execution Architecture	100%	Chamod	Built `ControlledExecutorService` with 5s timeout, subprocess

Subsystem / Feature	Contribution %	Author	Key Technical Work
			sandbox, stdout/stderr capture, & risk block
Command Safety & Risk Engine	100%	Chamod	Built regex risk classifier (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`) & confirmation locks
LLM Provider Abstraction	100%	Chamod	Built `LlmProvider` Java interface, `GeminiLlmProvider`, and `MockLlmProvider` fallback
Command Error Diagnosis Engine	100%	Chamod	Created error trace analyzer (`/api/tutor/command/diagnose`) & fix generator
MongoDB Command History UI & API	100%	Chamod	Built `CommandHistory` document entity, repository, single item delete, & search UI drawer
Total Contribution Weight	**~30%**	**Chamod**	**Architected & Implemented AI Command Assistance Subsystem**